

Smart Student Management System

Python Essentials – Project Report

Project Type	Command-Line Python Application
Course	Python Essentials
Student Name	Utkarsh Tarsolia
Registration / Roll No.	26BCE10641
College	VIT Bhopal University
Academic Year	2026–27
GitHub Repository	https://github.com/utkarsh15713/Smart-Student-Management-System

Submitted as part of the project evaluation requirements.

2. Introduction

The Smart Student Management System is a Python-based command-line application developed to manage student academic records. It stores student details and subject marks and provides options to add, view, search, update and delete records. It also performs basic academic analysis such as total marks, percentage, grade, pass/fail status, class statistics, top performers and subject-wise averages.

The application is divided into separate Python modules so that the main program, student operations, marks calculation, analysis, validation, utility functions and data storage can be maintained separately. Student records are stored in a JSON file, which allows the data to be loaded again when the application is started.

Main technologies used:

- Python 3
- JSON file storage
- VS Code
- Git and GitHub
- Command-line interface

3. Problem Statement

Managing student records manually can be time-consuming. Searching for a student, updating marks, calculating percentages and comparing class performance can also require repeated manual work.

The purpose of this project is to provide a simple command-line system that stores student information and marks in one place and performs common academic calculations automatically.

Project Objectives

- Provide a simple way to manage student records.
- Reduce repeated manual calculation of total and percentage.
- Provide grade and pass/fail information.
- Provide basic class and subject-level analysis.
- Keep records available using JSON file storage.
- Use modular Python programming concepts in a practical project.

4. Functional Requirements

The functional requirements describe what the system should do for the user.

ID	Module	Requirement
FR01	Add Student	Accept student details and subject marks and create a new record.
FR02	View Students	Display all stored student records.
FR03	Search Student	Find a student using roll number.
FR04	Update Student	Modify details or marks of an existing student.
FR05	Delete Student	Remove an existing student record.
FR06	Student Performance	Calculate/display total, percentage, grade and pass/fail status.
FR07	Class Statistics	Provide basic class-level performance information.
FR08	Top 3 Performers	Sort students by percentage and display the top three.
FR09	Subject-wise Analysis	Calculate average marks for each subject.
FR10	Data Storage	Save and load records using a JSON file.

5. Non-functional Requirements

ID	Requirement	Description
NFR01	Usability	Simple menu-driven command-line interface.
NFR02	Reliability	Invalid input should be handled without unexpected program termination.
NFR03	Maintainability	Separate modules make individual parts easier to change.
NFR04	Data Persistence	Records remain available after closing and reopening the program.
NFR05	Portability	Runs on a system with Python 3 without special hardware.
NFR06	Performance	Basic operations should complete quickly for a normal dataset.

6. System Architecture

The system follows a simple modular architecture. The user interacts with main.py through the command line. The main module controls the menu and calls the appropriate functions. Student operations are handled by student.py, mark calculations by marks.py, analysis by analysis.py, input checks by validation.py and JSON persistence by storage.py.

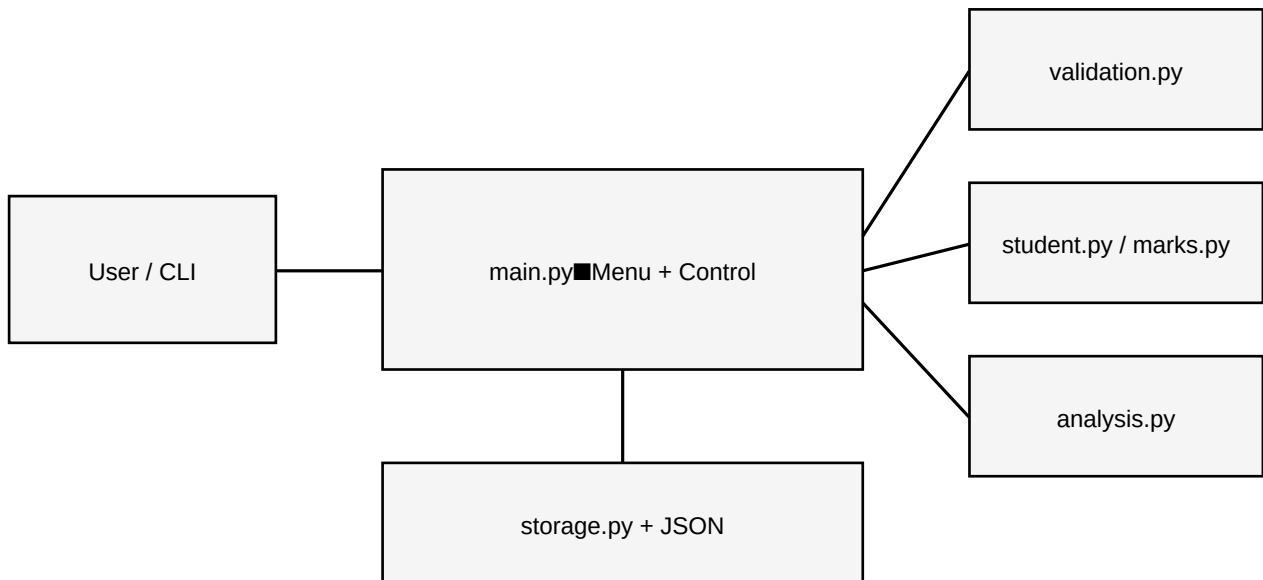


Figure 1: High-level system architecture

7. Design Diagrams

7.1 Use Case Diagram

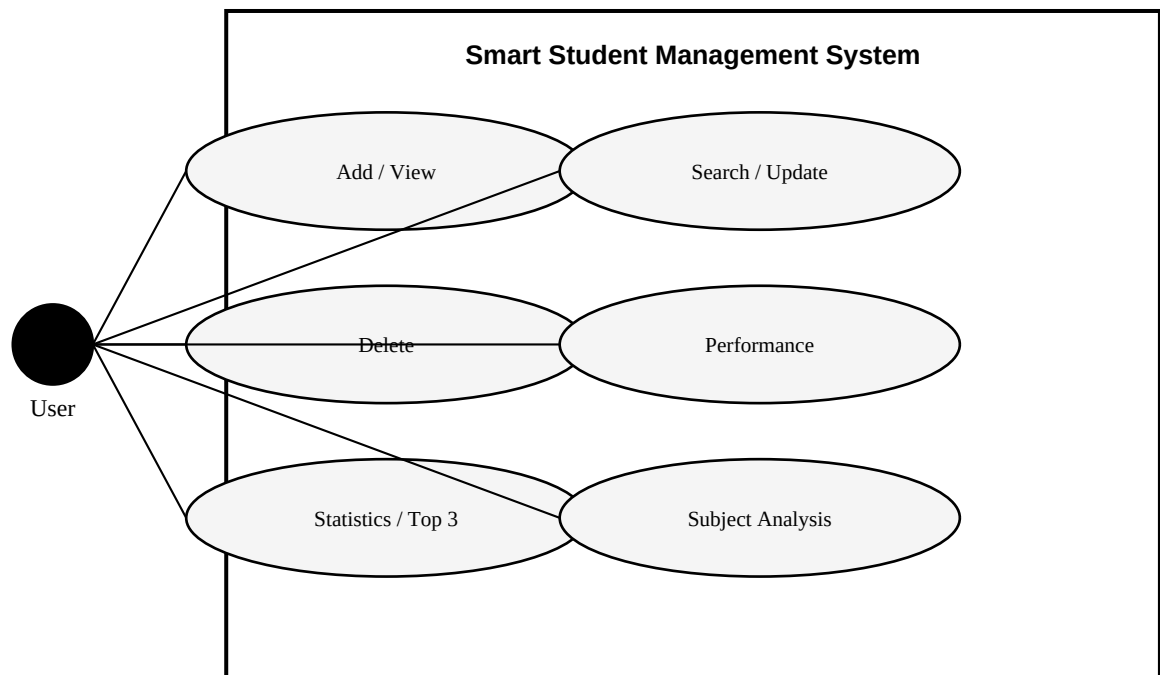


Figure 2: Use case diagram

7.2 Workflow Diagram

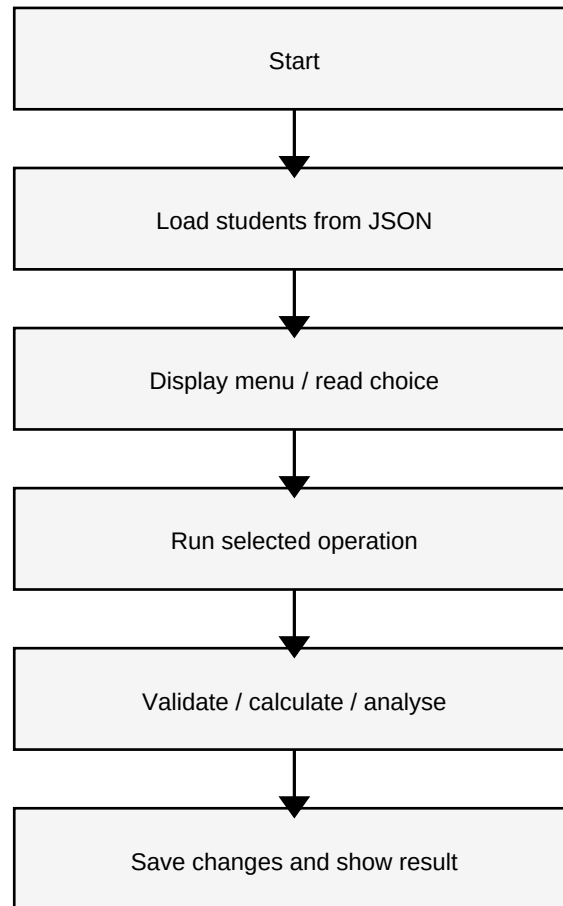


Figure 3: Main workflow

7.3 Sequence Diagram

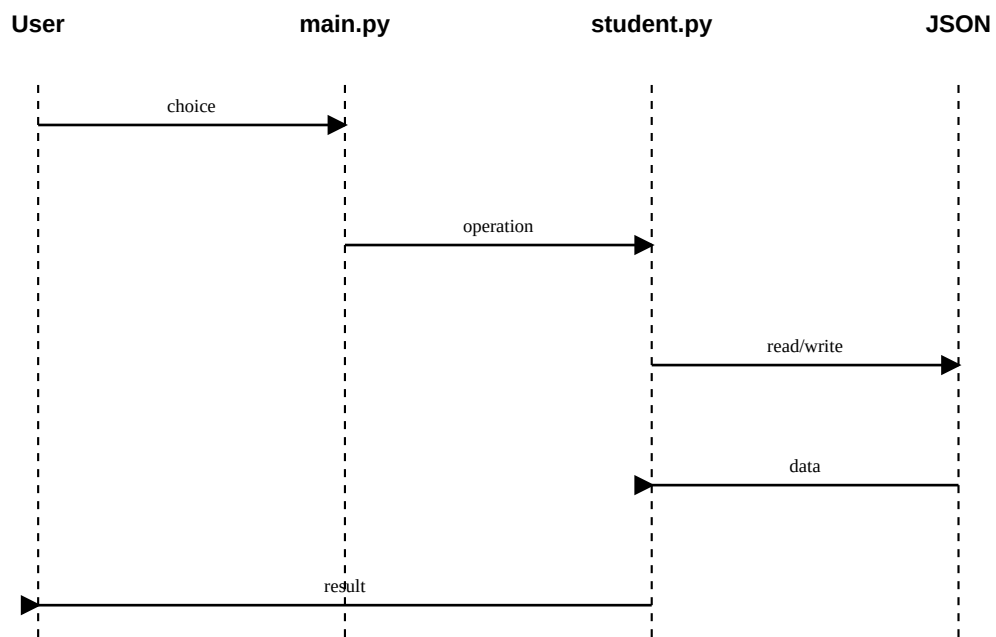


Figure 4: Example interaction sequence

7.4 Class / Component Diagram

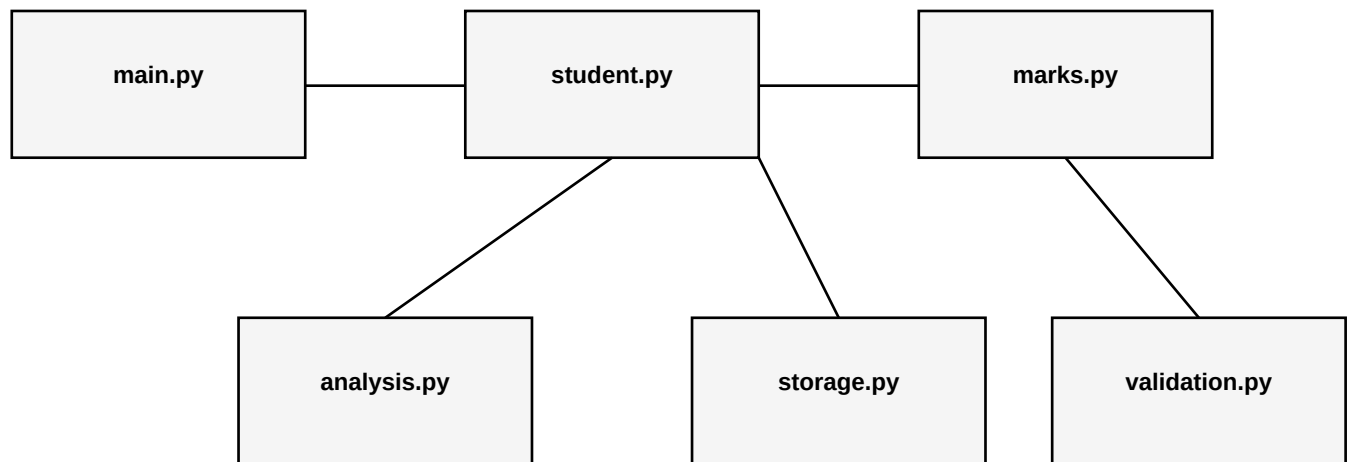


Figure 5: Module/component relationships

7.5 ER Diagram

The project uses JSON rather than a relational database. Therefore, this is a simplified data-model representation of the student record and its marks.

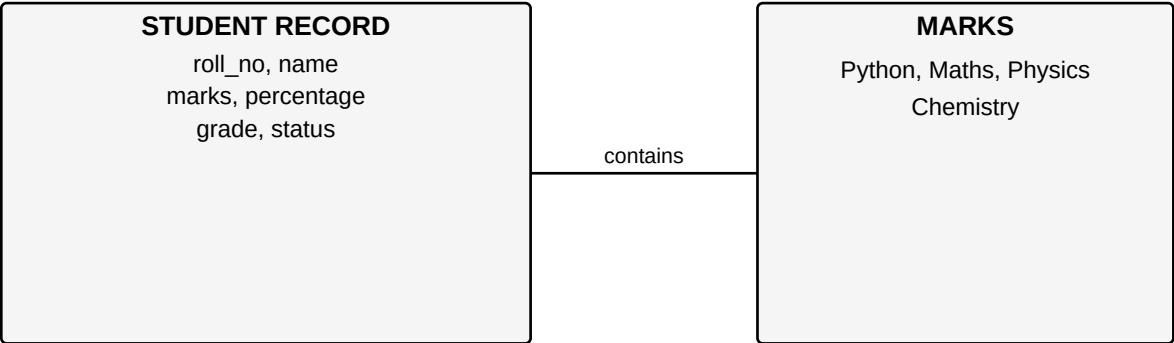


Figure 6: Simplified data model

8. Design Decisions & Rationale

Decision	Reason
Command-line interface	Keeps the project simple and demonstrates Python fundamentals directly.
Modular Python files	Separates responsibilities and makes the code easier to understand and maintain.
JSON storage	Easy to read and use without requiring an external database.
Roll number as identifier	Provides a simple way to find, update and delete a student record.
Input validation	Helps prevent invalid marks and incorrect inputs.
Sorting for Top 3	Uses percentage to compare records and display the highest three.

9. Implementation Details

The project is implemented using multiple Python files. Each file has a specific responsibility.

File	Purpose
main.py	Menu, user choices and overall program flow.
student.py	Add, view, search, update and delete student records.
marks.py	Total, percentage, grade and pass/fail calculations.
analysis.py	Performance, class statistics, top performers and subject analysis.
storage.py	Load and save records in data/students.json.
validation.py	Input and range validation.
utils.py	Header, menu, pause and exit functions.

Sample student data used during testing

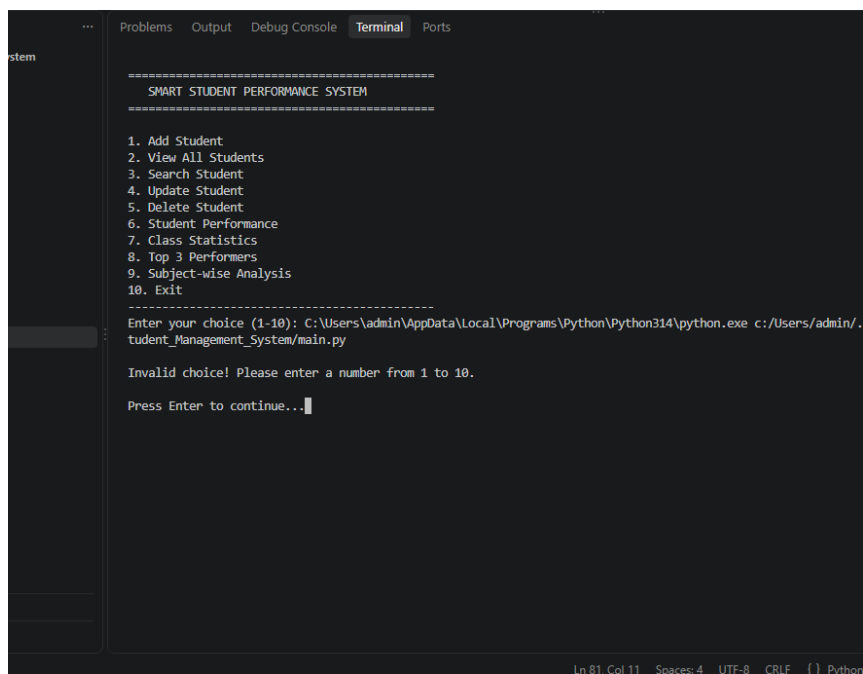
Roll No.	Name	Python	Maths	Physics	Chemistry
101	Rahul	92	88	90	85
102	Aman	84	79	86	81
103	Rohan	76	82	74	78
104	Arjun	68	72	65	70
105	Manish	88	91	84	89
106	Kunal	59	64	61	67

10. Screenshots / Results

The application was tested from the command line using sample student records. The following results were observed during testing. Terminal screenshots from the running application are shown below.

Feature	Observed Result
Student records	Sample records were stored in data/students.json.
Performance	For Rahul (Roll 101): total marks 355, percentage 88.75%, grade A, status PASS.
Top 3 Performers	Sorted by percentage: Rahul 88.75%, Manish 88.00%, Aman 82.50%.
Subject Analysis	Averages: Maths 79.33%, Chemistry 78.33%, Python 78.83%, Physics 76.67%. Strongest: Maths. Weakest: Physics.
Validation	Invalid inputs were handled with validation/error messages.

Screenshot 1 – Main Menu



```
=====
SMART STUDENT PERFORMANCE SYSTEM
=====

1. Add Student
2. View All Students
3. Search Student
4. Update Student
5. Delete Student
6. Student Performance
7. Class Statistics
8. Top 3 Performers
9. Subject-wise Analysis
10. Exit
=====

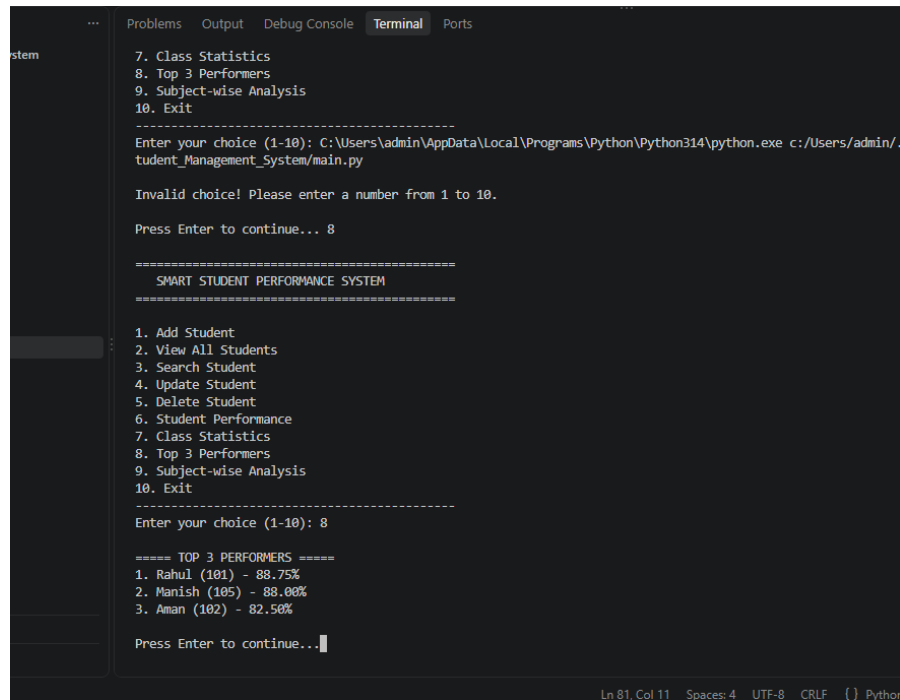
Enter your choice (1-10): C:\Users\admin\AppData\Local\Programs\Python\Python314\python.exe c:/Users/admin/.
tudent_Management_System/main.py

Invalid choice! Please enter a number from 1 to 10.

Press Enter to continue...
```

Figure 7: Main menu of the Smart Student Performance System showing options 1–10. An invalid input (full Python path) is rejected with a validation message.

Screenshot 2 – Top 3 Performers



```
7. Class Statistics
8. Top 3 Performers
9. Subject-wise Analysis
10. Exit
-----
Enter your choice (1-10): C:\Users\admin\AppData\Local\Programs\Python\Python314\python.exe c:/Users/admin/.
tudent_Management_System/main.py

Invalid choice! Please enter a number from 1 to 10.

Press Enter to continue... 8

=====
SMART STUDENT PERFORMANCE SYSTEM
=====

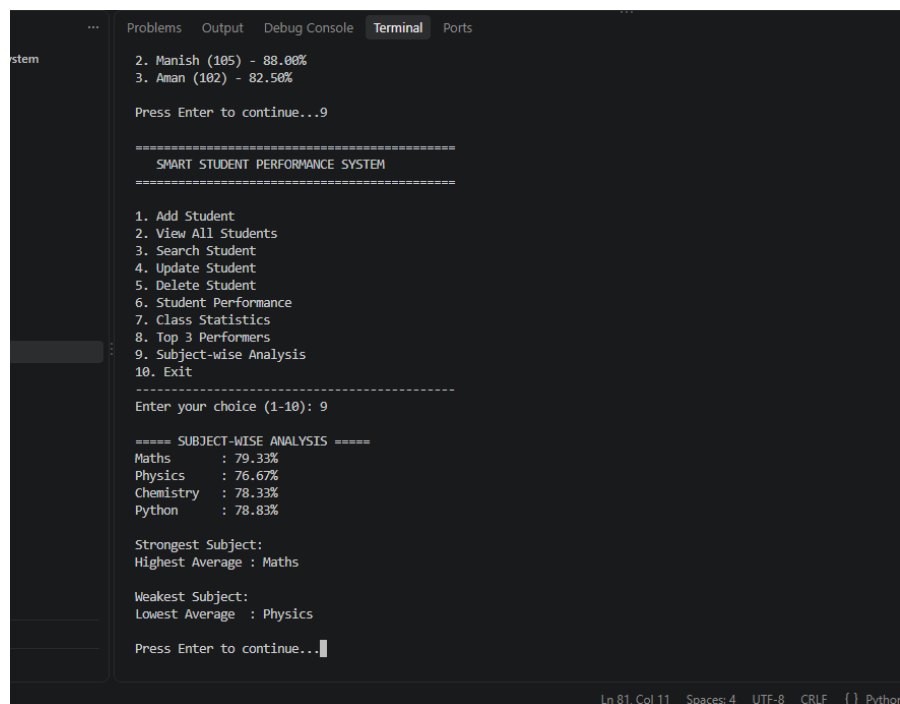
1. Add Student
2. View All Students
3. Search Student
4. Update Student
5. Delete Student
6. Student Performance
7. Class Statistics
8. Top 3 Performers
9. Subject-wise Analysis
10. Exit
-----
Enter your choice (1-10): 8

===== TOP 3 PERFORMERS =====
1. Rahul (101) - 88.75%
2. Manish (105) - 88.00%
3. Aman (102) - 82.50%

Press Enter to continue... |
```

Figure 8: Top 3 Performers output (Option 8). Students ranked by percentage: Rahul (101) – 88.75%, Manish (105) – 88.00%, Aman (102) – 82.50%.

Screenshot 3 – Subject-wise Analysis



```
2. Manish (105) - 88.00%
3. Aman (102) - 82.50%

Press Enter to continue...9

=====
SMART STUDENT PERFORMANCE SYSTEM
=====

1. Add Student
2. View All Students
3. Search Student
4. Update Student
5. Delete Student
6. Student Performance
7. Class Statistics
8. Top 3 Performers
9. Subject-wise Analysis
10. Exit
-----
Enter your choice (1-10): 9

===== SUBJECT-WISE ANALYSIS =====
Maths      : 79.33%
Physics    : 76.67%
Chemistry  : 78.33%
Python     : 78.83%

Strongest Subject:
Highest Average : Maths

Weakest Subject:
Lowest Average  : Physics

Press Enter to continue... |
```

Figure 9: Subject-wise Analysis (Option 9). Average marks per subject and identification of strongest subject (Maths – 79.33%) and weakest subject (Physics – 76.67%).

11. Testing Approach

Testing was performed by running the command-line application and checking each major menu option with valid and invalid inputs. The main aim was to verify that records are correctly stored, retrieved and modified and that the analysis functions produce the expected output.

Test Case	Input / Action	Expected / Observed Result	Status
Add Student	Enter valid student details	Record created	Pass
View Students	View all records	Stored records displayed	Pass
Search Student	Search by roll number	Matching record displayed	Pass
Update Student	Change existing record	Updated values displayed	Pass
Delete Student	Delete existing roll number	Record removed	Pass
Student Performance	Open performance option	Total, percentage, grade/status shown	Pass
Top 3 Performers	Use option 8	Top three sorted by percentage	Pass
Subject Analysis	Use subject analysis	Subject averages displayed	Pass
Invalid Input	Enter invalid values	Validation/error message shown	Pass

12. Challenges Faced

- Organising the project into separate Python files while keeping the modules connected correctly.
- Handling user input and checking that marks stay within a valid range.
- Making sure student data is saved and loaded correctly from the JSON file.
- Calculating percentage and grade consistently for every student record.
- Sorting student records correctly for the Top 3 Performers feature.
- Testing different menu options and invalid inputs to make the program more reliable.

Working through these issues helped in understanding how individual Python functions and files can be combined into a complete application.

13. Learnings & Key Takeaways

- Improved understanding of functions and modular programming.
- Used lists and dictionaries to represent student data.
- Practised file handling using JSON.
- Used sorting and basic calculations for data analysis.
- Learned the importance of input validation and error handling.
- Understood how to structure a small Python project using multiple files.
- Practised using Git and GitHub for project version control.
- Learned how to document a software project using diagrams and a report.

14. Future Enhancements

- Add a graphical user interface or web interface.
- Add login and role-based access for students and teachers.
- Use SQLite or another database for larger datasets.
- Generate printable student report cards.
- Add attendance and assignment tracking.
- Add charts for visual performance analysis.
- Add export options such as CSV or PDF.
- Add more advanced analytics and performance trends.

15. References

- Python 3 documentation and standard library concepts used in the project.
- Course material for Python Essentials.
- VITyarthi project instructions provided for the project evaluation.
- GitHub repository: <https://github.com/utkarsh15713/Smart-Student-Management-System>

Project repository structure: main.py, student.py, marks.py, analysis.py, storage.py, validation.py, utils.py, README.md, statement.md and data/students.json.

End of Report